

OE-EC704A — Web Technology

Semester VII • 3L:0T:0P • 3 Credits

Module 4 Exam Notes: Interfaces and Packages

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Interfaces	2
1.1	Syntax and Implementation	2
1.2	Interface Additions in Java 8 and 9	2
2	Packages	3
3	Access Control Mechanism	3
4	Dynamic Method Lookup	4

1. Interfaces

An interface is a reference type in Java that is similar to a class but contains only constants, method signatures, default methods, static methods, and nested types.

Interface Concept

An **Interface** defines a contract that implementing classes must satisfy. It specifies *what* a class should do, but not *how* it does it.

- Declared using the `interface` keyword.
- All variables declared inside an interface are implicitly `public`, `static`, and `final` (constants).
- All methods without a body are implicitly `public` and `abstract`.

1.1 Syntax and Implementation

A class implements an interface using the `implements` keyword. It must provide concrete overrides for all abstract methods declared in the interface.

```
interface Flyable {
    int SPEED_LIMIT = 300; // Implicitly public static final
    void fly();             // Implicitly public abstract
}

class Airplane implements Flyable {
    @Override
    public void fly() { // Must be declared public
        System.out.println("Airplane is flying at high altitude.");
    }
}
```

1.2 Interface Additions in Java 8 and 9

To support backward compatibility and clean API design, Java has expanded what an interface can contain:

- **Default Methods (Java 8):** Declared using the `default` keyword. They provide a default concrete implementation, allowing developers to add new methods to interfaces without breaking existing implementing classes.
- **Static Methods (Java 8):** Concrete methods bound to the interface itself. They cannot be overridden by implementing classes.
- **Private Methods (Java 9):** Used to encapsulate common helper logic shared among default/static methods within the same interface.

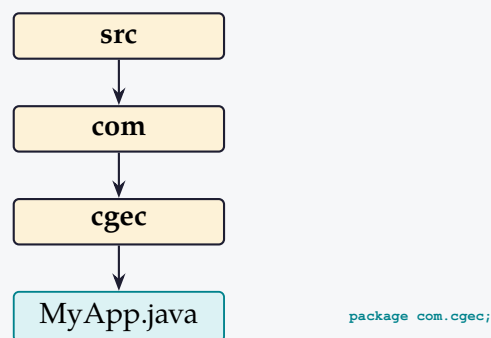
2. Packages

A package is a grouping of related classes, interfaces, and subpackages, providing namespace management and access protection.

Java Packages

- **Directory Mapping:** Packages map directly to the folder structure on disk. For example, a class in package `com.cgec.webtech` must reside in a folder path `com/cgec/webtech/`.
- **Namespace Clashing Prevention:** Ensures class name uniqueness by grouping them under a domain namespace.
- **The `package` Keyword:** Declared as the very first non-comment statement in a source file.
- **The `import` Keyword:** Used to make classes from other packages accessible within the current file.

Java Package Structure Mapping to File System



3. Access Control Mechanism

Java uses access modifiers to control visibility of classes, constructors, variables, and methods.

- **`private`:** Visible only within the declaring class. It provides the highest encapsulation.
- **Default (Package-Private):** Visible to all classes in the same package. No keyword is specified.
- **`protected`:** Visible to all classes in the same package, and to subclasses in different packages.
- **`public`:** Accessible from any class anywhere in the application.

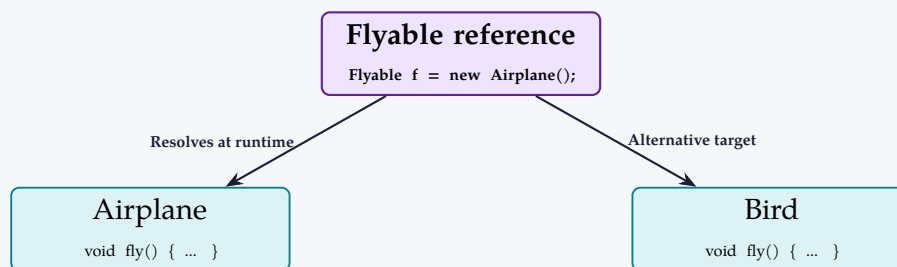
4. Dynamic Method Lookup

Dynamic Method Lookup (or Interface-based Dynamic Binding) is the runtime mechanism by which the JVM resolves calls to interface methods.

Interface Callbacks

When an interface reference calls a method, the JVM looks up the actual runtime class of the referenced object, retrieves its dynamic method table, and invokes the concrete overridden method. This is the foundation of loose coupling and polymorphic callbacks.

Interface Callback Mechanism



Quick Revision — Module 4

Interfaces and Packages | OE-EC704A

Interface & Package Declarations

Concept	Syntax and Example Usage
Interface Definition	<code>interface InterfaceName { void methodSignature(); }</code>
Implementation	<code>class ClassName implements InterfaceName { ... }</code>
Default Method	<code>default void log() { System.out.println("Default Log"); }</code>
Static Method	<code>static int add(int a, int b) { return a + b; }</code>
Packages & Imports	Core syntax headers
Package Declaration	<code>package com.cgec.webtech; // must be first line</code>
Class Import	<code>import java.util.ArrayList;</code>
Wildcard Import	<code>import java.util.*;</code>
Static Import	<code>import static java.lang.Math.PI;</code>

One-Line Definitions

- **Interface:** A reference type declaring method signatures and constants, defining a contract.
- **Package:** A directory structure grouping related classes and interfaces.
- **Default Method:** An interface method containing a default body, declared with the `default` keyword.
- **Access Modifier:** Keywords controlling the accessibility level of classes and class members.
- **Dynamic Method Lookup:** Runtime resolution of interface method calls based on actual object instance type.
- **Encapsulation:** Bundling class state and behavior while restricting direct access using `private`.
- **Namespace:** A naming container resolving name conflicts across different components.
- **Multiple Inheritance:** Implementing multiple interfaces to inherit behaviors from multiple sources.

Frequently Asked / Viva Questions

Q1. Can an interface be instantiated? Why?

No. An interface does not have constructors and contains abstract method signatures without implementations, so it cannot represent concrete runtime state.

Frequently Asked / Viva Questions (contd.)**Q2. Why does Java allow multiple inheritance using interfaces but not classes?**

Classes hold state (instance variables). If a class inherits from multiple parent classes, duplicate variable names cause conflicts. Since interfaces hold no state variables (only constants) and historically had no implementations, there is no duplicate implementation ambiguity.

Q3. What is the difference between an Interface and an Abstract Class?

An abstract class can contain instance variables (state) and constructors, whereas an interface can only contain public static final constants. A class can extend only one class but implement multiple interfaces.

Q4. What are default methods and why were they introduced in Java 8?

Default methods have concrete bodies declared with the `default` keyword inside an interface. They allow adding new methods to interfaces without breaking existing classes that implement the interface.

Q5. What is a functional interface?

An interface containing exactly one abstract method. It can be annotated with `@FunctionalInterface` and is used for Lambda expressions (e.g., `Runnable`, `Comparable`).

Q6. Can we declare an interface method as `private` or `protected`?

Interface abstract methods must be `public`. However, starting from Java 9, we can declare `private` and `private static` helper methods inside interfaces. `protected` is never allowed.

Q7. What is the default access modifier in an interface?

All fields are implicitly `public static final`. All abstract methods are implicitly `public abstract`.

Q8. What is a static import in Java?

A static import allows importing static members (variables and methods) of a class directly (e.g., `import static java.lang.Math.sqrt;`). This allows invoking them without prepending the class name.

Q9. What is the difference between `package` and `import` statements?

The `package` statement declares which package/directory the current class belongs to. The `import` statement tells the compiler where to find other classes referenced in the file.

Q10. How does the JVM resolve name conflicts between classes in different imported packages?

If a class with the same name exists in two imported packages (e.g., `java.util.Date` and `java.sql.Date`), the compiler forces the developer to use the fully qualified class name in code to resolve ambiguity.

Java Access Modifier Visibility Matrix

Modifier	Same Class	Same Package	Subclass	Everywhere
private	Yes	No	No	No
Default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes